DelosDB Unified Storage I/O Abstraction

Design Proposal: DelosPageVolume

DelosDB Engineering · June 2026 · Confidential Draft

ABSTRACT
This proposal defines a unified page-level I/O abstraction layer for the DelosDB storage engine. It draws on verified source-level analysis of H2 Database, PostgreSQL, and MariaDB/InnoDB, on three peer-reviewed VLDB papers (2023–2026), and on the JDK 21/22/24 platform roadmap (Project Loom, JEP 491, Project Panama/FFM API). The central deliverable is the DelosPageVolume interface: a narrow, five-method contract that eliminates JDK 21 virtual-thread pinning, enables fully in-memory test execution, provides deterministic fault injection for crash-recovery proofs, and opens a clear upgrade path to memory-mapped I/O and — via the JDK 22 Foreign Function & Memory API — kernel-bypass I/O (io_uring) without JNI.

1 Background and Motivation

DelosDB is a Java 21 fork of Apache Derby that replaces Derby's monolithic storage layer with a capability-segregated table-access contract (DelosTableAccess / DelosFilterableTableAccess / DelosMutableTableAccess) and a fully independent MVCC storage engine (delosdb-storage-mvcc). The native Derby execution path — GenericResultSetFactory, DelosTableScanResultSet, DelosInsertResultSet, DelosDeleteResultSet, DelosUpdateResultSet — was completed in Phase F and verified green across the full Derby language test suite.

The MVCC engine's durable layer (MvccPageFile, MvccPageMutationLog, MvccTransactionOutcomeLog, MvccDurableIndexStore) currently uses synchronized java.nio.channels.FileChannel methods. This design has three concrete deficiencies on the JDK 21 platform:

- **Virtual-thread pinning (JDK 21–23).** Performing blocking I/O inside a synchronized method pins the virtual thread to its carrier OS thread for the full duration of the system call. Every MvccPageFile.readPage(), writePage(), and force() call triggers this condition, capping concurrency to the number of physical CPU cores regardless of how many virtual threads are scheduled.
- **Persistent pinning on force() even in JDK 24.** JEP 491 (Synchronize Virtual Threads without Pinning, delivered in JDK 24) removes pinning for most synchronized blocking operations. However, channel.force(true) is a native system call (fsync). Native calls continue to pin virtual threads in JDK 24; the force() method remains problematic without structural change.
- **No in-memory test mode.** Every MVCC test writes to real disk, paying full fsync cost. There is no mechanism to run MvccPageFile backed by off-heap memory — the pattern that H2's OffHeapStore and Derby's VFMemoryStorageFactory provide for their respective engines. Absence of in-memory test mode also prevents deterministic crash-recovery proofs for MvccTransactionOutcomeLog and the MvccHistoryPrunedException path.

2 Analysis of Existing Systems

Source-level analysis of three production database systems and one research engine establishes what has already been proven to work at scale.

2.1 H2 Database — FileStore and OffHeapStore

H2's MVStore layer defines FileStore<C> as an abstract class parameterised by its chunk type. SingleFileStore wraps a java.nio.channels.FileChannel for on-disk operation; OffHeapStore implements the same contract using a TreeMap<Long, ByteBuffer> backed entirely by off-heap memory. OffHeapStore.open() initialises the in-memory store; force() is a no-op. MVStore.Builder accepts a FileStore instance via its config map, making the storage backend a first-class, construction-time parameter. Source inspection confirmed that OffHeapStore is used in both test and production contexts (four test uses, five production uses), validating the pattern as more than a test convenience.

Lesson. *A narrow abstract class with read, write, allocate, sync, and open/close methods — implemented by both a disk-backed and an off-heap-backed subclass — is proven at production scale.*

2.2 PostgreSQL — smgr Dispatch Table

PostgreSQL's storage manager (src/backend/storage/smgr/smgr.c) defines the f_smgr struct: a C function-pointer table with 20 callbacks covering open, close, create, exists, unlink, extend, zero-extend, prefetch, read (synchronous and asynchronous via PgAioHandle), write, writeback, nblocks, truncate, immedsync, and registersync. The dispatch table smgrsw[] contains exactly one entry (md.c, the magnetic-disk manager) with the comment 'we only have md.c at present'. The async AIO entry points (smgr_startreadv, pgaio_io_set_target_smgr) were added in recent releases as PostgreSQL moves toward kernel-bypass I/O.

Lessons. *(1) Build the dispatch table only when two real implementations exist; PostgreSQL has carried one-entry infrastructure for decades. (2) The fsync abstraction (smgr_immedsync, smgr_registersync) is a first-class, separately named operation in a mature engine. (3) The async read entry point (smgr_startreadv) demonstrates the correct seam for future io_uring integration.*

2.3 MariaDB / InnoDB — Engine-Owned I/O

MariaDB's handler contract (sql/handler.h, 218 virtual methods) is entirely silent on I/O. Each storage engine owns its I/O layer independently: InnoDB uses os0file.h (pread/pwrite/async AIO, 18 implementation files); MyISAM uses the shared mysys library (my_read/my_write) plus per-table function pointers (MI_INFO.file_read / MI_INFO.file_write) for I/O policy swapping; RocksDB ships its own RocksDB Env abstraction. innodb_flush_method exposes fsync policy (O_DIRECT, O_DSYNC, fsync, nosync) as a runtime configuration knob within InnoDB only.

Lesson. *Engines with genuinely different I/O requirements need not share a common VFS. The handler contract covers SQL semantics; I/O is entirely the engine's own concern. DelosDB should follow MariaDB's precedent: the DelosTableAccess contract covers row-access semantics; I/O is DelosMVCC's internal concern.*

2.4 LeanStore — Unified I/O Backend (VLDB 2023–2024)

LeanStore (Leis et al., PVLDB 2024) is a high-performance OLTP storage engine optimised for NVMe SSDs and many-core CPUs. Its I/O backend provides a unified abstraction over libaio, io_uring, and SPDK, implementing RAID-0 data striping in the DBMS itself rather than relying on the Linux software RAID stack. The backend also abstracts the details of asynchronous I/O submission and completion, enabling the engine to saturate tens of millions of IOPS without kernel scheduler interference.

Lesson. *The correct interpretation of LeanStore for DelosDB is not that DelosDB should bypass the OS kernel — LeanStore's thesis is specifically about NVMe saturation at scale, which is not DelosDB's current operating regime. The transferable lesson is that the sync / force boundary is the correct seam for future I/O backend substitution. DelosPageVolume.force() is that seam.*

3 JDK Platform Capabilities: JDK 21 through JDK 25

The JDK 21→25 roadmap provides a clear, staged improvement path for database storage I/O on the JVM. Each stage is independently valuable and incrementally adoptable.

JDK	Feature	Relevance to DelosDB Storage
21	Virtual Threads (JEP 444)	readPage/writePage pin carrier threads inside synchronized blocks
21	FileChannel + MemorySegment preview	Memory-mapped page access with Arena lifetime control
22	FFM API stable (JEP 454)	MemorySegment replaces ByteBuffer; deterministic off-heap lifetime via Arena
24	JEP 491: Sync without pinning	synchronized no longer pins VTs — except for native calls (fsync)
24+	ReentrantLock best practice	Eliminates force() pinning on all JDK versions including 21–23
22+	FFM Linker (Project Panama)	io_uring calls from pure Java via MethodHandle, no JNI boilerplate
25+	FFM ecosystem maturity	Community liburing bindings; io_uring IORING_OP_FSYNC via FFM

The critical JDK 21 finding is that MvccPageFile's six synchronized methods pin virtual threads during every disk I/O call. The immediate, correct fix — independent of any abstraction work — is to replace synchronized with java.util.concurrent.locks.ReentrantLock. This eliminates pinning on JDK 21/22/23 and ensures force() remains safe after JDK 24's JEP 491 lands, since force() contains a native call that JEP 491 cannot unmount regardless of lock type.

4 Proposed Interface: DelosPageVolume

The proposed interface is derived directly from MvccPageFile's existing five public methods. It introduces no new operations. The only additions are: an explicit SyncPolicy enum (replacing the implicit always-force behaviour), a syncPolicy() accessor, and the AutoCloseable contract (already implemented by MvccPageFile).

```
public interface DelosPageVolume extends AutoCloseable {

    /** Read the page at the given page identifier. */
    MvccPage readPage(MvccPageId id) throws IOException;

    /** Write a complete page to the volume. */
    void writePage(MvccPage page) throws IOException;

    /** Allocate a new page of the given type and return it. */
    MvccPage allocatePage(int pageType) throws IOException;

    /** Return the total number of pages in this volume. */
    long pageCount() throws IOException;

    /**
     * Durability boundary.
     * SyncPolicy.FULL         - channel.force(true) / real fsync
     * SyncPolicy.METADATA_ONLY - channel.force(false)
     * SyncPolicy.NONE         - no-op (in-memory / test implementations)
     *
     * SyncPolicy is set at construction, not per-call, to prevent
     * accidental durability bypass in production code.
     */
    void force() throws IOException;
```

```
    SyncPolicy syncPolicy();

    @Override
    void close() throws IOException;

    enum SyncPolicy { FULL, METADATA_ONLY, NONE }
}
```

MemorySegment (java.lang.foreign, JDK 22) replaces ByteBuffer as the data representation inside MvccPage. This change eliminates the 2 GB address ceiling (ByteBuffer uses int indexing; MemorySegment uses long), enables deterministic off-heap deallocation via Arena rather than relying on garbage collection, and opens the zero-copy path via FileChannel.map(READ_WRITE, offset, size, arena) for the mapped implementation described in Section 5.

5 Planned Implementations

FileChannelPageVolume

JDK 21 — Production

Wraps the existing MvccPageFile behaviour exactly. ReentrantLock replaces synchronized throughout. Page buffers allocated via Arena.ofConfined() (MemorySegment). force() calls channel.force(policy == FULL). This is a refactor of existing code, not a rewrite.

MappedPageVolume

JDK 21 preview / JDK 22 stable — Production candidate

Uses FileChannel.map(READ_WRITE, 0, maxBytes, arena) to create an Arena-scoped MemorySegment backed by the page file. readPage() performs a direct MemorySegment.get() at the computed offset — zero kernel copies. writePage() performs a direct MemorySegment.set(). force() calls segment.force() to flush dirty pages. Requires a pre-defined maximum file size; suitable for bounded research workloads. Benchmarking against FileChannelPageVolume determines which becomes the default.

OffHeapPageVolume

JDK 21 — Test and research

Implements the full DelosPageVolume contract using a TreeMap indexed by page ID, backed by Arena.ofShared() for thread-safe access. force() is a no-op (SyncPolicy.NONE enforced at construction). Directly mirrors H2's OffHeapStore pattern but uses MemorySegment rather than ByteBuffer, eliminating the 2 GB ceiling even in large test scenarios. Enables fully in-memory MVCC test execution with no disk I/O.

FaultInjectingPageVolume

JDK 21 — Crash-recovery research

Decorates any DelosPageVolume implementation. Accepts a FaultSchedule that specifies: failOnWrite(n) to throw IOException on the n-th write, failOnForce(n) to simulate fsync failure, tearPage(n) to corrupt the n-th page written (simulating a torn write). Enables deterministic crash-recovery proof tests for MvccHistoryPrunedException, MvccTransactionOutcomeLog recovery, and the unresolved-outcome recovery path.

IoUringPageVolume

JDK 22+ via FFM — Future lane

Uses the JDK 22 Foreign Function and Memory API (JEP 454) to call liburing functions directly from Java via Linker.nativeLinker() and MethodHandle downcall handles — no JNI required. Submits batched readv/writev/fsync operations via the io_uring submission queue. force() submits an IORING_OP_FSYNC entry and returns before kernel completion, enabling asynchronous durability. Built only when FileChannelPageVolume benchmarks show I/O as the bottleneck at the target working-set size.

6 The JVM Native Bridge: Foreign Function & Memory API

The Foreign Function and Memory API (Project Panama, JEP 454, finalised in JDK 22) is the JVM's modern replacement for JNI. It provides three capabilities directly relevant to storage I/O:

- **MemorySegment** — a safe, Arena-scoped view of off-heap memory with 64-bit addressing, deterministic deallocation, and spatial bounds checking (IndexOutOfBoundsException on overrun, not a JVM segfault). Replaces ByteBuffer for all page buffer allocations.
- **Linker.nativeLinker()** — creates MethodHandle instances that call native C functions by name, with full type safety expressed via FunctionDescriptor. The JIT compiler can optimise these calls to near-JNI performance. No .so/.dll shim required for system libraries already present on the host (liburing, libc).
- **Arena** — manages native memory lifetime. Arena.ofConfined() for single-threaded page I/O. Arena.ofShared() for concurrent page-buffer pools. Try-with-resources ensures deterministic release without GC pressure.

The concrete path to io_uring from Java without JNI boilerplate is demonstrated in the literature (Jasny et al., PVLDB 2026; Houlborg et al., CIDR 2026) and in open-source examples (davidvlijmincx.com/posts/async-io-with-java-and-panama). The pattern: SymbolLookup.libraryLookup() to load liburing.so, FunctionDescriptor.of(JAVA_INT, ADDRESS) to describe io_uring_submit(), Arena.ofShared() to allocate the io_uring sq/cq ring buffers. IoUringPageVolume would encapsulate this entirely behind the DelosPageVolume interface, invisible to callers.

7 Impact and Scope

Four files in delosdb-storage-mvcc hold a MvccPageFile field: PageBackedMvccTable, MvccIndexStore, MvccDurableIndexStore, and PageBackedMvccTableStore. Each requires one field-type change (MvccPageFile → DelosPageVolume) and one construction-site update. No other production code changes. The interface addition is backwards-compatible: FileChannelPageVolume is a drop-in wrapper around the current MvccPageFile logic.

The scope boundary is explicit: DelosPageVolume covers MvccPageFile only. MvccPageMutationLog (append-only WAL), MvccTransactionOutcomeLog (outcome journal), and the text-line mutation log have different I/O access patterns and receive parallel interfaces when independently justified. Forcing all four durable structures into a single interface would produce a lowest-common-denominator abstraction that serves none of them well.

OUT OF SCOPE
The upper-layer provider dispatch table (analogous to PostgreSQL's smgrsw[]) is NOT part of this proposal. That layer is built only after the K1 feasibility gate confirms that Derby heap can honestly become a live Delos provider. PostgreSQL's own source shows the cost of building dispatch infrastructure before the second implementation exists: smgrsw[] has carried one entry and a 'we only have md.c at present' comment for decades.

8 Implementation Sequence

Phase	Deliverable	JDK	Duration
Immediate	Replace synchronized with ReentrantLock in MvccPageFile Eliminates virtual-thread pinning on all JDK versions	21	1–2 days

Phase	Deliverable	JDK	Duration
Short-term	Extract DelosPageVolume interface Implement FileChannelPageVolume (wraps existing code) Implement OffHeapPageVolume (MemorySegment + Arena) Implement FaultInjectingPageVolume Migrate MvccPage from ByteBuffer to MemorySegment	21/22	1 week
Near-term	Implement MappedPageVolume Benchmark vs FileChannelPageVolume Extend A49/A50 recovery proofs with FaultInjectingPageVolume	22	1 week
Future lane	IoUringPageVolume via FFM Linker Only if benchmark identifies FileChannel as bottleneck	22+	When justified

9 Explicit Exclusions

No common VFS across providers

MariaDB demonstrates that engines with different I/O requirements do not need a shared VFS. InnoDB, MyISAM, and RocksDB each own their I/O layer and coexist without interference. DelosDB follows the same precedent: DelosPageVolume is MVCC-internal.

No URL-scheme FilePath layer

H2's FilePath abstraction (8+ scheme implementations including disk, mem, nio-mem, encrypted, split, async, zip, rec) is the correct reference if encryption, split files, or protocol-routed storage become requirements. That layer is not built until those requirements are real.

No io_uring by default

The PVLDB 2026 paper (Jasny et al.) demonstrates that io_uring's benefits are workload-dependent: specifically beneficial at high IOPS with concurrent batched submissions. DelosDB's current single-threaded synchronized FileChannel generates one I/O at a time. IoUringPageVolume is a named future implementation, not a default.

No LeanStore-style kernel bypass

LeanStore bypasses the OS kernel (via SPDK) specifically to saturate multiple NVMe SSDs at tens of millions of IOPS. This is not DelosDB's current operating regime. DelosPageVolume's force() method is the correct seam for that direction, if and when the workload justifies it.

No provider-level dispatch table

The upper layer (provider selection, analogous to PostgreSQL's f_smgr smgrsw[]) is deferred until K1 confirms that Derby heap can honestly become a live Delos provider alongside delos_mvcc. PostgreSQL's source is the evidence for why premature dispatch infrastructure is a long-lived maintenance burden.

R References

[1] Leis, V. (2024). LeanStore: A High-Performance Storage Engine for NVMe SSDs. Proc. VLDB Endow. 17(12), 4536–4545. https://doi.org/10.14778/3685800.3685915

[2] Alhomssi, A. & Leis, V. (2023). Scalable and Robust Snapshot Isolation for High-Performance Storage Engines. PVLDB 16(6), 1426–1438. https://www.vldb.org/pvldb/vol16/p1426-alhomssi.pdf

[3] Haas, G. & Leis, V. (2023). What Modern NVMe Storage Can Do, and How to Exploit It: High-Performance I/O for High-Performance Storage Engines. Proc. VLDB Endow. 16(9), 2090–2102. https://doi.org/10.14778/3598581.3598584

[4] Jasny, M., El-Hindi, M., Ziegler, T., Leis, V. & Binnig, C. (2026). io_uring for High-Performance DBMSs: When and How to Use It. PVLDB 19(1). https://arxiv.org/abs/2512.04859

[5] Houlborg, E. et al. (2026). Flexible I/O for Database Management Systems with xNVMe. CIDR 2026. https://vldb.org/cidrdb/papers/2026/p6-houlborg.pdf

[6] Leis, V. & Dietrich, C. (2024). Cloud-Native Database Systems and Unikernels: Reimagining OS Abstractions for Modern Hardware. PVLDB 17(8), 2115–2122.

[7] OpenJDK JEP 444: Virtual Threads. Java 21. https://openjdk.org/jeps/444

[8] OpenJDK JEP 491: Synchronize Virtual Threads without Pinning. Java 24. https://openjdk.org/jeps/491

[9] OpenJDK JEP 454: Foreign Function and Memory API. Java 22 (finalised). https://openjdk.org/jeps/454

[10] OpenJDK JEP 352: Non-Volatile Mapped Byte Buffers. https://openjdk.org/jeps/352

[11] H2 Database Engine, source: org.h2.mvstore.FileStore, org.h2.mvstore.OffHeapStore, org.h2.mvstore.SingleFileStore. https://github.com/h2database/h2database

[12] PostgreSQL source: src/backend/storage/smgr/smgr.c, src/include/storage/smgr.h. https://github.com/postgres/postgres

[13] MariaDB server source: sql/handler.h, storage/innobase/include/os0file.h, storage/myisam/myisamdef.h. https://github.com/MariaDB/server

[14] Apache Derby / DelosDB: delosdb-storage-mvcc, MvccPageFile.java. https://github.com/ggeorg/delosdb

[15] JDK 21 Release Notes — FileChannel and Virtual Thread behaviour. https://www.oracle.com/java/technologies/javase/21-relnote-issues.html

[16] Vlijmincx, D. (2025). Async IO with Java and Panama: Unlocking the Power of io_uring. https://davidvlijmincx.com/posts/async-io-with-java-and-panama/